CH 3 — System Modelling

Luke Lu • 2026-11-03

We would like to visualize the design of a software system

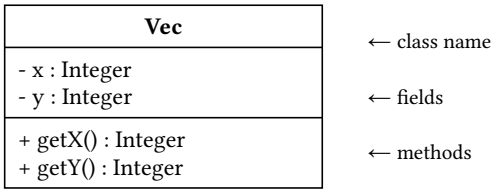
1. Identify key entities/abstraction/classes
2. Relationships between entities

Use Unified Modelling Language

Modeling classes

It can be represented as a box with 3 sections

Example



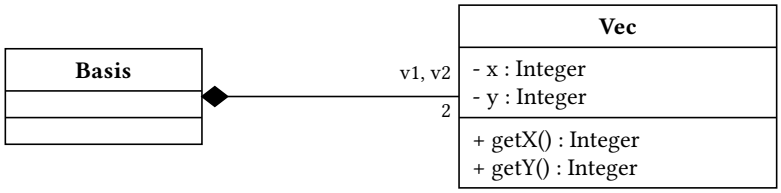
`- ...` means it is `private`

`+ ...` means it is `public`

Relationship 1: Composition/OWNS

```
class Basis{
  Vec v1, v2;
};
```

`Basis` is composed/owns of 2 `Vec` objects



The **filled diamond** ◆ sits on the **owner** (`Basis`); the role names and multiplicity are written at the **owned** end (`Vec`).

A OWNS-A B implies:

- if A is destroyed, B is destroyed
- if A is copied, B is copied

When a class has fields of objects, those objects are owned by the class

Sometimes fields thare pointers to objects might also have a OWNS-A relationship

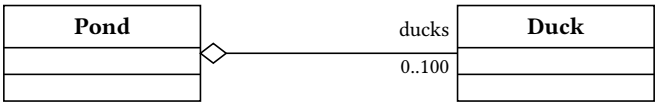
Relationship 2: Aggregation/HAS:

- Aggregation is never represented using fields that are objects.
- Fields that are pointers DO NOT automatically represnet a HAS-A relationship

A HAS-A B implies:

- If A is destroyed, B lives on

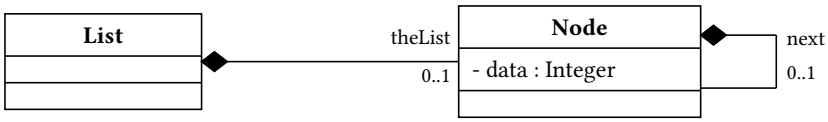
```
class Pond{
  Duck *ducks[100];
}
```



The **hollow diamond** ◇ sits on the container (`Pond`). Multiplicities use `..` (e.g. `0..100`, `0..*`).

Nodes of Lists

```
class List {
  Node *theList;   // List OWNS its nodes
};
class Node {
  int data;
  Node *next;      // each Node OWNS the next one
};
```



Pointer fields here are still OWNS-A: the `List` is responsible for deleting its nodes, and each `Node` for deleting `next`.

Relationshp 3: Specialization/Inheritance/IS

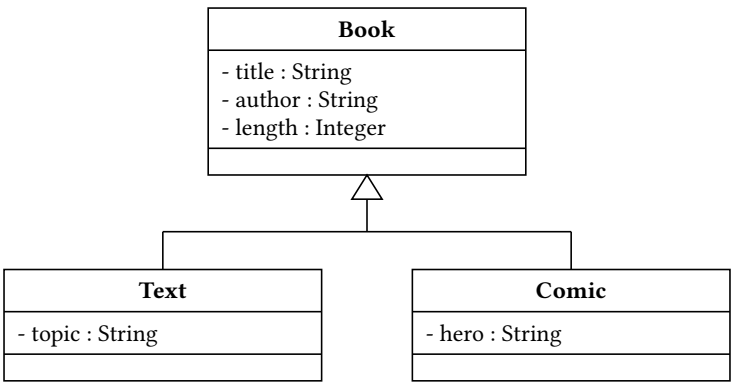
Consider example:

- Book: title, author, length
  - Text: title, author, length, topic
  - COmic: title, author, length, hero

Observe:

Text IS-A Book specialized with an extra topic field,

Comic IS-A Book specialized with an extra hero field



The **hollow triangle** △ points **at the superclass**. Subclasses only list their **new** fields; inherited ones are not repeated.

`Book` is a Parent/Base/Super class

`Comic` and `Text` are Children/Derived/Sub class which inherits all members of `Book` including private, public, and Methods

Any methods that you could call a `Book` object you can call on a `Text` object

Code can be found "lectures/19-inheritance/examples"

```
class Book {
  string title, author;
  int length;
  public:
    Book(string title, string author, int length):
    title{title}, author{author}, length{length}
};

class Text: public Book{
  string topic;

};

class Comic: public Book{
  string hero;
};
```

Here `Text` inherited 3 private fields: `title, author, length`

But `private` means private that is

A `Text` object reserves space for `title, author, length, topic`

Even though `Text` objects have space for these `private` inherited fields, `Text` class cannot access these fields

```
//INVALID
Text::Text(string title, string author, int length, string topic):
    title{title}, author{author}, length{length}, topic{topic};
```

The above INVALID code has 3 errors:

1. Inherited fields are `private`
2. MIL can only initialize fields it declared
3. Steps for object construction of derived classes need superclass constructed first

NEW steps for constructing derived objects:

1. Space is reserved
2. Superclass part of the object is constructed
3. Subclass fields that are objects are constructed/Modeling
4. Constructor body runs

So we need a default constructor of `Book`, but since we wrote our own default constructor for `Book`, we need to call the custom constructor explcitly in `Text`

```
Text::Text(string title, string author, int length, string topic) :
 Book{title, author, length}, topic{topic} {}
```

`protected` members are accessible to inherited subclasses

```
class Book{
  protected:
    string author;
};

void Text::addAuthor(string auth){
  author+= auth;
}

int main(){
  Text t;
  //INVALID, requires public level
  t.author=...;
}
```

Here `Text` can access `authors` b/c it is protected in `Book`

> 💡 **Tip — Advice for choosing Level**
>
> 1. Keep fields private to guarantees invariants
> 2. We could provide `protected` mutators/accessors

Method Overriding

Imagine: I want to answer when different books are heavy.

`Book` is heavy if `lenght>200`

`Text` is heavy if `length>500`

`Comic` is heavy if `length>30`

Code found “lectures/inheritance/example2”

```
class Book{
  int length;
  public:
    int getLength() const {return length}
    bool isHeavy() const {return length > 200;}
};

bool Text:isHeavy() const {return getLength() > 500;}
bool Comic:isHeavy() const {return getLength() > 30;}
```

Method overriding: eplacing an inherited method with a new implementation of the method

```
Book b1{"a", "b", 50};
b1.isHeavy(); //Book::isHeavy() runs, gives out `false`
Comic c1{"c", "d", 40, "e"};
c1.isHeavy(); //Comic::isHeavt() runs, gives out `true`

Book b2 = c1; //LEGAL b/c Comic IS-A Book

b2.isHeavy(); //Book::isHeavy() runs, gives out `false`

//`Book` copy constructor ran, treating `c1` as a `Book`
```

Object slicing/coersion:

```
Superclass s = subObject;
```

The additional fields of `subObject` are dropped when `Superclass`'s copy constructor is called.

We want to avoid this.

```
Comic c1{"c", "d", 40, "e"};
Comic *pc {&c1};

pc->isHeavy(); //Comic::isHeavy() runs, gives out `true`

Book *pb{&c1}; //VALID b/c `Comic` is a `Book`, the reverse is INVALID
//No slicing occurred b/c the pointer is simply pointing to the location of `c1`
pb->isHeavy(); //Book::isHeavy() runs, `false` in Java it would have been `true`
```

C++ looks at the declared type of the object/pointer to determine which method to run (by default)